3.Fichas de diseño de componentes (IEEE 1016-2009):

Modulo de Membresias se organiza en componentes funcionales que interactúan entre sí mediante interfaces definidas. La interfaz de usuario solicita operaciones a la lógica de negocio, que valida las reglas correspondientes y accede a SQL Server mediante la capa de datos. Para el procesamiento de pagos se contempla, además, la comunicación con una pasarela externa.

Los componentes principales son:

1. Autenticación e inicio de sesión.
2. Panel principal o Dashboard.
3. Gestión de clientes.
4. Gestión de planes.
5. Gestión de membresías.
6. Gestión de pagos y pasarela de pago.
7. Reportes y consultas.

. Fichas de diseño de componentes

3.1. Componente de autenticación e inicio de sesión

Identificador: SDD-CMP-001

Nombre: Módulo de autenticación.

Propósito: Permitir el acceso seguro al sistema mediante la validación de las credenciales del usuario y la identificación de sus permisos.

Responsabilidades:

- Capturar el usuario o correo electrónico y la contraseña.
- Validar que las credenciales estén completas.
- Verificar las credenciales mediante el servicio de autenticación.
- Iniciar la sesión cuando las credenciales sean válidas.
- Mostrar mensajes de error cuando la autenticación falle.
- Restringir el acceso a las funciones según los permisos asignados.

Atributos principales:

Atributo	Tipo	Descripción
IdUsuario	**INT**	**Identificador del usuario.**
NombreUsuario	**VARCHAR**	**Nombre de usuario registrado.**

Correo	VARCHAR	Correo asociado a la cuenta, si corresponde.
HashContrasena	VARCHAR	Representación segura de la contraseña almacenada.
Rol	VARCHAR	Rol asignado, por ejemplo, Administrador.
Estado	BIT	Indica si la cuenta está habilitada.

Interfaces:

- `IniciarSesion(usuario, contrasena)`: solicita la autenticación y devuelve el resultado.
- `CerrarSesion()`: finaliza la sesión activa.
- `ValidarPermisos(rol, recurso)`: verifica si el usuario puede acceder a una funcionalidad.

Dependencias:

- Formulario de inicio de sesión.
- Servicio de autenticación.
- Base de datos SQL Server, tabla `Usuarios`.
- Módulo de autorización.

Manejo de excepciones:

- Credenciales incorrectas: mostrar un mensaje de acceso denegado.
- Campos vacíos: solicitar que se completen los datos.
- Error de conexión con la base de datos: informar que el servicio no está disponible.
- Cuenta deshabilitada: impedir el acceso.
- Error inesperado: registrar el incidente y mostrar un mensaje controlado sin exponer información sensible.

Consideraciones de diseño: Las contraseñas no deben almacenarse en texto plano. Se utilizará un algoritmo de hash de contraseñas con salt. Si se emplea JWT para acceder a servicios externos, el token deberá validarse y administrarse de forma segura.

3.2. Componente del panel principal (Dashboard)

Nombre: Panel principal de FitControl.

Propósito: Presentar un resumen del estado del gimnasio mediante indicadores, actividad reciente y accesos a los módulos principales.

Responsabilidades:

- Mostrar la cantidad de clientes activos.
- Presentar los ingresos registrados durante el período seleccionado.
- Mostrar las membresías próximas a vencer.

- Informar sobre nuevas altas de membresías.
- Mostrar las actividades recientes del sistema.
- Permitir la navegación hacia clientes, planes, membresías y pagos.

Atributos principales:

Atributo	Tipo	Descripción
TotalClientesActivos	**INT**	**Cantidad de clientes con membresía activa.**
IngresosPeriodo	**DECIMAL(10,2)**	**Total de ingresos del período consultado.**
MembresiasPorVencer	**INT**	**Membresías próximas a finalizar.**
NuevasMembresias	**INT**	**Membresías registradas en el período.**
ActividadReciente	**LIST**	**Operaciones recientes que se muestran al usuario.**

Interfaces:

- `ObtenerResumen()`: recupera los indicadores del panel.
- `ObtenerActividadReciente()`: consulta las últimas operaciones registradas.
- `NavegarModulo(nombreModulo)`: abre el módulo seleccionado.

Dependencias:

- Módulos de clientes, membresías y pagos.
- Base de datos SQL Server.
- Servicio de consultas y agregación de indicadores.

Manejo de excepciones:

- Error al consultar indicadores: mostrar un aviso y permitir reintentar.
- Ausencia de registros: mostrar valores en cero o un mensaje de información.
- Error de conexión: mantener la interfaz disponible y notificar que los datos no pudieron actualizarse.
- Datos desactualizados: permitir actualizar el panel mediante una nueva consulta.

Consideraciones de diseño: Los indicadores deben calcularse a partir de los registros existentes en la base de datos. Los ingresos deberán considerar únicamente los pagos que se encuentren confirmados según las reglas del sistema.

3.3. Componente de gestión de clientes

Nombre: Módulo de clientes.

Propósito: Administrar la información personal y el estado de los clientes registrados en el gimnasio.

Responsabilidades:

- Registrar nuevos clientes.
- Consultar y listar clientes.
- Buscar clientes por nombre u otros criterios.
- Modificar información personal.
- Consultar el estado de las membresías asociadas.
- Permitir el acceso a la información de cada cliente.

Atributos principales:

Atributo	Tipo	Descripción
IdCliente	**INT**	**Identificador único del cliente.**
Nombre	**VARCHAR**	**Nombre del cliente.**
Apellido	**VARCHAR**	**Apellido del cliente.**
Telefono	**VARCHAR**	**Número de contacto.**
Correo	**VARCHAR**	**Correo electrónico.**
FechaRegistro	**DATE**	**Fecha de registro en el gimnasio.**
Estado	**VARCHAR**	**Estado del cliente según las reglas del sistema.**

Interfaces:

- `RegistrarCliente(cliente)`: guarda un nuevo cliente.
- `ListarClientes(filtros)`: recupera clientes según los filtros aplicados.
- `BuscarCliente(idCliente)`: obtiene la información de un cliente.
- `ActualizarCliente(cliente)`: modifica sus datos.
- `ConsultarMembresias(idCliente)`: obtiene las membresías asociadas al cliente.

Dependencias:

- Formulario de clientes.
- Componente de membresías.
- Capa de lógica de negocio.
- Base de datos SQL Server, tabla `Clientes`.

Manejo de excepciones:

- Campos obligatorios vacíos: impedir el registro.
- Correo con formato inválido: solicitar su corrección.
- Cliente duplicado según los criterios de unicidad definidos: rechazar el registro.
- Cliente inexistente: mostrar un mensaje informativo.
- Error al guardar o actualizar: informar el resultado y evitar mostrar una operación como exitosa si no se confirmó.

Consideraciones de diseño: Las operaciones deberán validar los datos antes de enviarlos a la base de datos y utilizar consultas parametrizadas para reducir riesgos de inyección SQL.

3.4. Componente de gestión de planes

Nombre: Módulo de planes.

Propósito: Administrar los planes comerciales que el gimnasio ofrece a sus clientes.

Responsabilidades:

- Registrar nuevos planes.
- Consultar y listar los planes disponibles.
- Modificar precios, duración y descripción.
- Activar o desactivar planes.
- Proporcionar información a los módulos de membresías y pagos.

Atributos principales:

Atributo	Tipo	Descripción
IdPlan	**INT**	**Identificador único del plan.**
NombrePlan	**VARCHAR**	**Nombre comercial del plan.**
Descripcion	**VARCHAR**	**Detalle de los beneficios ofrecidos.**
Precio	**DECIMAL(10,2)**	**Precio del plan.**
DuracionDias	**INT**	**Duración del plan en días.**
Estado	**BIT**	**Indica si el plan está disponible.**

Interfaces:

- `RegistrarPlan(plan)`: crea un plan.
- `ListarPlanes()`: obtiene los planes registrados.
- `ActualizarPlan(plan)`: modifica los datos del plan.
- `ConsultarPlan(idPlan)`: recupera los datos de un plan.

- `CambiarEstadoPlan(idPlan, estado)`: habilita o deshabilita un plan.

Dependencias:

- Formulario de planes.
- Módulo de membresías.
- Módulo de pagos.
- Base de datos SQL Server, tabla `Planes`.

Manejo de excepciones:

- Precio negativo o duración no válida: rechazar el registro.
- Plan inexistente: informar que no se encontró.
- Plan asociado a membresías existentes: impedir su eliminación física cuando afecte la integridad referencial; permitir desactivarlo según las reglas del negocio.
- Error de persistencia: informar que la operación no se completó.

Consideraciones de diseño: Los precios y las duraciones deberán validarse antes de registrar un plan. Los cambios en los planes no deberán modificar retroactivamente los pagos ya realizados.

3.5. Componente de gestión de membresías

Nombre: Módulo de membresías.

Propósito: Controlar la contratación, vigencia y estado de las membresías de los clientes del gimnasio.

Responsabilidades:

- Registrar una membresía para un cliente y un plan.
- Calcular o validar las fechas de inicio y vencimiento.
- Consultar las membresías registradas.
- Actualizar los datos permitidos de una membresía.
- Identificar membresías activas, próximas a vencer, vencidas o canceladas.
- Relacionar la membresía con los pagos correspondientes.

Atributos principales:

Atributo	Tipo	Descripción
IdMembresia	**INT**	**Identificador único de la membresía.**
IdCliente	**INT**	**Referencia al cliente titular.**
IdPlan	**INT**	**Referencia al plan contratado.**
FechaInicio	**DATE**	**Fecha de inicio de la membresía.**

FechaFin	DATE	Fecha de vencimiento.
Estado	VARCHAR(20)	Activa, Vencida o Cancelada.

Interfaces:

- `RegistrarMembresia(membresia)`: registra una membresía.
- `ListarMembresias(filtros)`: consulta membresías por cliente, estado o fecha.
- `BuscarMembresia(idMembresia)`: obtiene una membresía específica.
- `ActualizarMembresia(membresia)`: modifica los campos permitidos.
- `CalcularFechaFin(fechaInicio, duracionDias)`: determina la fecha de finalización según la duración del plan.
- `ActualizarEstadoMembresia(idMembresia)`: verifica y actualiza el estado según las fechas y las reglas del negocio.

Dependencias:

- Módulo de clientes.
- Módulo de planes.
- Módulo de pagos.
- Capa de acceso a datos, por ejemplo, `MembresiaDAL`.
- Capa de lógica de negocio, por ejemplo, `MembresiaBLL`.
- Base de datos SQL Server, tabla `Membresias`.

Manejo de excepciones:

- Cliente inexistente: impedir la asignación de la membresía.
- Plan inexistente o desactivado: impedir la contratación.
- Fecha final anterior o igual a la fecha inicial: rechazar los datos.
- Membresía inexistente: informar que no se encontró.
- Error de base de datos: cancelar la operación y conservar la consistencia de los registros.
- Membresía vencida: actualizar o presentar el estado correspondiente sin confundirlo con un pago pendiente.

Consideraciones de diseño: La vigencia de la membresía y el estado del pago son conceptos distintos. Una membresía no deberá considerarse pagada únicamente por haber sido creada. Si el negocio exige confirmar el pago antes de activar la membresía, ambas operaciones deberán coordinarse para evitar activaciones sin cobro confirmado.

3.6. Componente de gestión de pagos y pasarela de pago

Nombre: Módulo de pagos.

Propósito: Registrar los pagos de las membresías y comunicarse con una pasarela de pago externa para procesar transacciones electrónicas.

Responsabilidades:

- Mostrar el monto que debe pagar el cliente.
- Asociar el pago con el cliente y la membresía correspondiente.
- Enviar la solicitud de pago a la pasarela externa.
- Procesar la respuesta de aprobación o rechazo.
- Registrar el identificador de la transacción.
- Mantener el historial y el estado de los pagos.
- Notificar al módulo de membresías cuando un pago quede confirmado.

Atributos principales:

Atributo	Tipo	Descripción
IdPago	**INT**	**Identificador interno del pago.**
IdMembresia	**INT**	**Membresía a la que corresponde el pago.**
Monto	**DECIMAL(10,2)**	**Importe que se pretende cobrar.**
Moneda	**VARCHAR(3)**	**Moneda de la transacción, por ejemplo, GTQ.**
FechaPago	**DATETIME**	**Fecha y hora del pago.**
Estado	**VARCHAR(20)**	**Pendiente, Aprobado o Rechazado.**
TransactionId	**VARCHAR**	**Identificador de la transacción externa, cuando exista.**
CardToken	**VARCHAR**	**Token de pago proporcionado para la transacción, si corresponde.**

Interfaces:

- `ProcesarPago(idMembresia, monto, moneda, cardToken)`: solicita el procesamiento de un pago.
- `ConsultarPago(idPago)`: recupera la información de un pago.
- `ConsultarEstadoTransaccion(transactionId)`: consulta el estado de una transacción.
- `RegistrarResultadoPago(resultado)`: guarda el resultado confirmado.
- `NotificarPagoConfirmado(idMembresia)`: comunica la confirmación al módulo de membresías.

Dependencias:

- Módulo de membresías.
- Base de datos SQL Server, tabla `Pagos`.
- Servicio de integración con la pasarela de pago externa.
- Servicio de autenticación y autorización.

- Conexión segura a Internet.

Manejo de excepciones:

- Monto inválido o negativo: rechazar la solicitud antes de enviarla.
- Membresía inexistente: impedir el cobro.
- Pago rechazado: registrar el resultado como rechazado cuando exista una respuesta confirmada.
- Tiempo de espera agotado: marcar el resultado como pendiente de verificación cuando no se sepa si la pasarela procesó el cobro.
- Error de conexión con la pasarela: informar que no fue posible confirmar la operación.
- Transacción duplicada: verificar la operación previa y evitar cobros repetidos mediante un mecanismo de idempotencia cuando el proveedor lo permita.
- Error al guardar el resultado: no informar que el pago quedó registrado correctamente; reconciliar la operación con la pasarela antes de reintentar un cobro.

Consideraciones de diseño: La integración se realizará mediante la API documentada del proveedor de pagos que se seleccione. El uso de `cardToken` representa el envío de un token de pago, no de los datos completos de la tarjeta. No se deberán almacenar números completos de tarjetas ni códigos de seguridad. Las credenciales del proveedor y los tokens deberán protegerse.

El endpoint `POST /payments/process` de la API FitControl podrá recibir `amount`, `currency` y `cardToken`, exigir autenticación Bearer cuando corresponda y devolver el identificador de transacción junto con su estado. El resultado deberá reflejar la respuesta real de la pasarela, no una aprobación simulada.

3.7. Componente de reportes y consultas

Propósito: Proporcionar información consolidada sobre clientes, membresías, ingresos y pagos para apoyar las actividades administrativas del gimnasio.

Responsabilidades:

- Consultar clientes registrados.
- Generar listados de membresías activas, vencidas y próximas a vencer.
- Consultar los pagos confirmados por período.
- Calcular indicadores e ingresos a partir de los registros almacenados.
- Presentar los resultados en pantalla y permitir su exportación cuando esa función esté implementada.

Atributos principales:

Atributo	Tipo	Descripción
TipoReporte	**VARCHAR**	**Tipo de reporte solicitado.**

FechaInicio	DATE	Inicio del período de consulta.
FechaFin	DATE	Fin del período de consulta.
TotalRegistros	INT	Número de registros obtenidos.
TotalIngresos	DECIMAL(10,2)	Total calculado de pagos confirmados.
FechaGeneracion	DATETIME	Momento en que se genera el reporte.

Interfaces:

- `GenerarReporte(tipo, fechaInicio, fechaFin)`: genera un reporte.
- `ConsultarIngresos(fechaInicio, fechaFin)`: calcula los ingresos confirmados.
- `ConsultarMembresiasPorEstado(estado)`: obtiene las membresías según su estado.
- `ExportarReporte(formato)`: exporta los resultados a un formato compatible, si está habilitado.

Dependencias:

- Módulos de clientes, membresías y pagos.
- Base de datos SQL Server.
- Componente de presentación de reportes.
- Biblioteca de exportación, si corresponde.

Manejo de excepciones:

- Rango de fechas inválido: solicitar un rango correcto.
- Ausencia de datos: mostrar un reporte vacío o un mensaje informativo.
- Error de consulta: informar que no se pudo generar el reporte.
- Error de exportación: notificar el problema y permitir intentar nuevamente.

Consideraciones de diseño: Los reportes deberán consultar información consistente y respetar los permisos del usuario. Los ingresos se calcularán a partir de pagos confirmados, excluyendo operaciones rechazadas o pendientes.

4. Interfaces y dependencias entre componentes

La interacción entre los módulos deberá seguir el orden lógico del negocio:

1. El usuario inicia sesión y obtiene acceso según sus permisos.
2. El panel principal consulta los indicadores necesarios.
3. El módulo de clientes proporciona la información del titular de la membresía.
4. El módulo de planes proporciona el precio y la duración contratados.
5. El módulo de membresías registra la contratación y controla su vigencia.
6. El módulo de pagos solicita la transacción a la pasarela y registra el resultado.

7. Los módulos de dashboard y reportes consultan los datos almacenados para mostrar información actualizada.

Las dependencias deberán mantenerse controladas. Por ejemplo, el módulo de reportes consultará los datos de pagos y membresías sin asumir la responsabilidad de modificarlos directamente.

5. Criterios generales de manejo de excepciones

Todos los componentes deberán cumplir con los siguientes criterios:

- Validar los datos antes de ejecutar operaciones de persistencia.
- Utilizar consultas parametrizadas para interactuar con SQL Server.
- Gestionar las excepciones sin mostrar detalles internos, contraseñas ni credenciales.
- Registrar los errores técnicos relevantes para facilitar el diagnóstico.
- Informar al usuario mediante mensajes comprensibles.
- Utilizar transacciones de base de datos cuando una operación implique varios cambios que deban confirmarse conjuntamente.
- Evitar que un error en un componente produzca registros inconsistentes en otros módulos.
- Distinguir entre errores de validación, errores de acceso, fallos de infraestructura y resultados de negocio, como un pago rechazado.